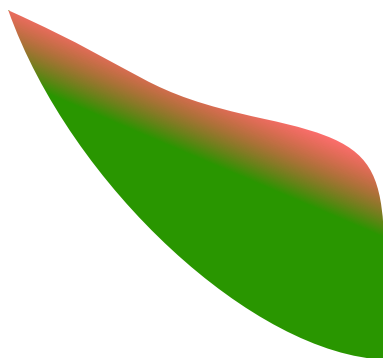




abaplint.app

Documentation

Table of Contents

1. Introduction	1
1.1. Requirements	1
1.2. Support	1
2. Quick Start	1
3. Configuration	2
3.1. Repository Access	2
3.2. Rule Configuration	2
3.3. App Configuration	3
3.3.1. Multiple Configurations	3
3.3.2. Experimental Features	3
3.3.3. Allowing no artifacts	4
3.3.4. Run merge check	4
3.3.5. Incremental mode	4
3.4. Dependent GitHub Actions	4
4. Annotations	6
5. Observations	7
5.1. New void type used	7
5.2. DB field changed	7
5.3. Target Rules	8
5.4. Parallel File Changes	8
5.5. Function module type changed or deleted	8
6. Suggestions	9
7. Target Rules	10
7.1. Configuration	10
7.2. Insights	10
8. Insights	11

8.1. Issues	11
8.2. Method Length	11
8.3. Method Complexity	11
8.4. Statement Compatibility	11
8.5. Void Types	12
8.6. Dependencies	12
8.7. Package Coupling	12
8.8. UML Class Diagrams	13
8.9. Disabled Rules	14
8.10. Repository Relations	14
8.11. Lines Over Time	14
8.12. Inheritance over time	14
8.13. Unit Tests	14
8.14. Lines vs Unit Tests	15
8.15. Average Complexity	15
8.16. Average Method Length	16
8.17. Target Rules	16
8.18. Class Length	16
8.19. Intra Class Call Graph	16
8.20. Object Types	17
8.21. Procedural vs OO	17
8.22. Table Accesses	18
8.23. API Usage	18
8.24. Object Classifications	18
8.25. Tree Map, Lines of Code	18
9. Cross Check	19
9.1. Example	19
10. Additional Information	21
10.1. Licensing	21
10.2. Privacy	21

Home	https://github.com/heliconialabs/docs.abaplint.app
License	Creative Commons
Build	2026-01-25 10:41:58 UTC

1. Introduction

abaplint.app is a cloud based tool for helping ABAP developers develop ABAP. The app can be used for static analysis and continuous integration, providing multiple configurable rules to support any development guidelines.

1.1. Requirements

A GitHub user or organization is required for use with the abaplint.app. The latest version of [abapGit](#) is required for serializing the ABAP artifacts, abapGit works on SAP_BASIS 702 and up.

1.2. Support

Users with paid plans can contact support@abaplint.app with questions, bugs and feature requests.

2. Quick Start

Perform the following steps to get started,

1. [Create GitHub User or Organization](#)
2. [Install the app from the marketplace](#)
3. [Install abapGit in the ABAP system](#)
4. [Push ABAP code to GitHub via abapGit](#)

abaplint.app is automatically triggered for each commit, or when a pull request is opened.

By default, abaplint will run all rules if there is no configuration found. Check the abaplint documentation for a [minimal syntax checking configuration](#).

3. Configuration

3.1. Repository Access

The administrator of a GitHub organization controls which repositories that abaplint.app can access via <https://github.com/apps/abaplint/installations/new>.

Only grant access to repositories where abaplint.app will be used,

Repository access

☐ **All repositories**
This applies to all current *and* future repositories.

☒ **Only select repositories**

Select repositories ▼

Selected 20 repositories.

larshp/abapOpenChecks	×
larshp/abapNTLM	×
larshp/abapOpenReview	×
larshn/abanGitServer	×

Save Cancel

Administrators can revoke or grant additional repository access at any time, note that the app permissions are limited to read and updating information in pull requests.

3.2. Rule Configuration

abaplint is configured via `/abaplint.json` in the git repository, if no configuration file is present, the default configuration is used, which have all rules enabled.

The default configuration can be found on playground.abaplint.app, or by running ``abaplint -d`` on command line.

If editing the configuration file via [vscode abaplint extension](#), the editor will know the format of the json configuration file and help discovering and updating rule configurations.

rules.abaplint.org lists and documents all rules which can be configured.

3.3. App Configuration

A file with one of the following filenames can optionally be placed in the root of the git directory,

- `abaplint-app.json`
- `abaplint-app.jsonc`
- `abaplint-app.json5`

Only the first file found will be taken into account.

The file can be used to configure abaplint.app features. The corresponding schema can be accessed at <https://schema.abaplint.app/schema.json>.

3.3.1. Multiple Configurations

By adding a `abaplint-app.json` file it is possible to have abaplint.app run multiple abaplint configurations. [Observations](#) and [suggestions](#) are reported only for the first configuration. Configurations are processed in the sequence as defined in the file,

```
{
  "configurations": {
    "default": {
      "filename": "./abaplint.jsonc"
    },
    "v740sp05": {
      "filename": "./abaplintonprem.jsonc"
    }
  }
}
```

When running with the above configuration, abaplint.app also analyzes the source code using `abaplintonprem.jsonc` and issues the result as "abaplint / v740sp05",



**Some checks were not successful**

1 failing and 2 successful checks

[Hide all checks](#)

	 abaplint Failing after 11s — 5 issues	Details
	 abaplint / observations Successful in 8s — 0 observations	Details
	 abaplint / v740sp05 — 0 issues	Details
	This branch has no conflicts with the base branch Merging can be performed automatically.	

3.3.2. Experimental Features

Experimental features are disabled by default, but can be enabled by setting

```
{
  "experimentalFeatures": true
}
```

These features change over time and are only documented when the feature becomes generally available.

3.3.3. Allowing no artifacts

By default, abaplint.app will report an error if no ABAP artifacts are found, this error can be disabled with:

```
{
  "noArtifactsOkay": true
}
```

3.3.4. Run merge check

NOTE | Experimental, this is work in progress and may change or be removed without notice.

When enabled, abaplint.app will analyze the code as it would look like after merging the pull request into the target branch. This will give results in pending pull requests, running the checks without the branches having to be up to date.

```
{
  "mergeCheck": true
}
```

3.3.5. Incremental mode

NOTE | Experimental, this is work in progress and may change or be removed without notice.

When enabled, abaplint.app will only analyze files changed in the pull request. This can be used to speed up the analysis for large repositories with many files. And useful for repositories with high number of legacy issues, where only new or changed files should be analyzed.

```
{
  "incremental": true
}
```

3.4. Dependent GitHub Actions

abaplint.app helps with static analysis, the next step in a CI pipeline is typically to run the unit tests. Github Actions can be used to trigger any logic after the static analysis completes.

Template and working example can be found at <https://github.com/heliconialabs/abaplint-app-dependent-action>

Example Github Action configuration,

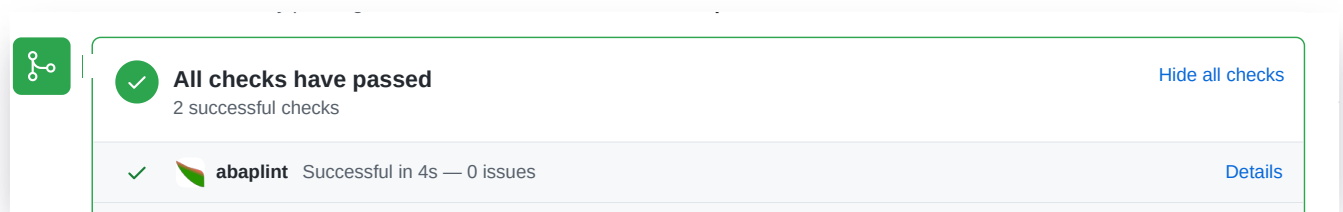
```
name: unit

on:
  pull_request

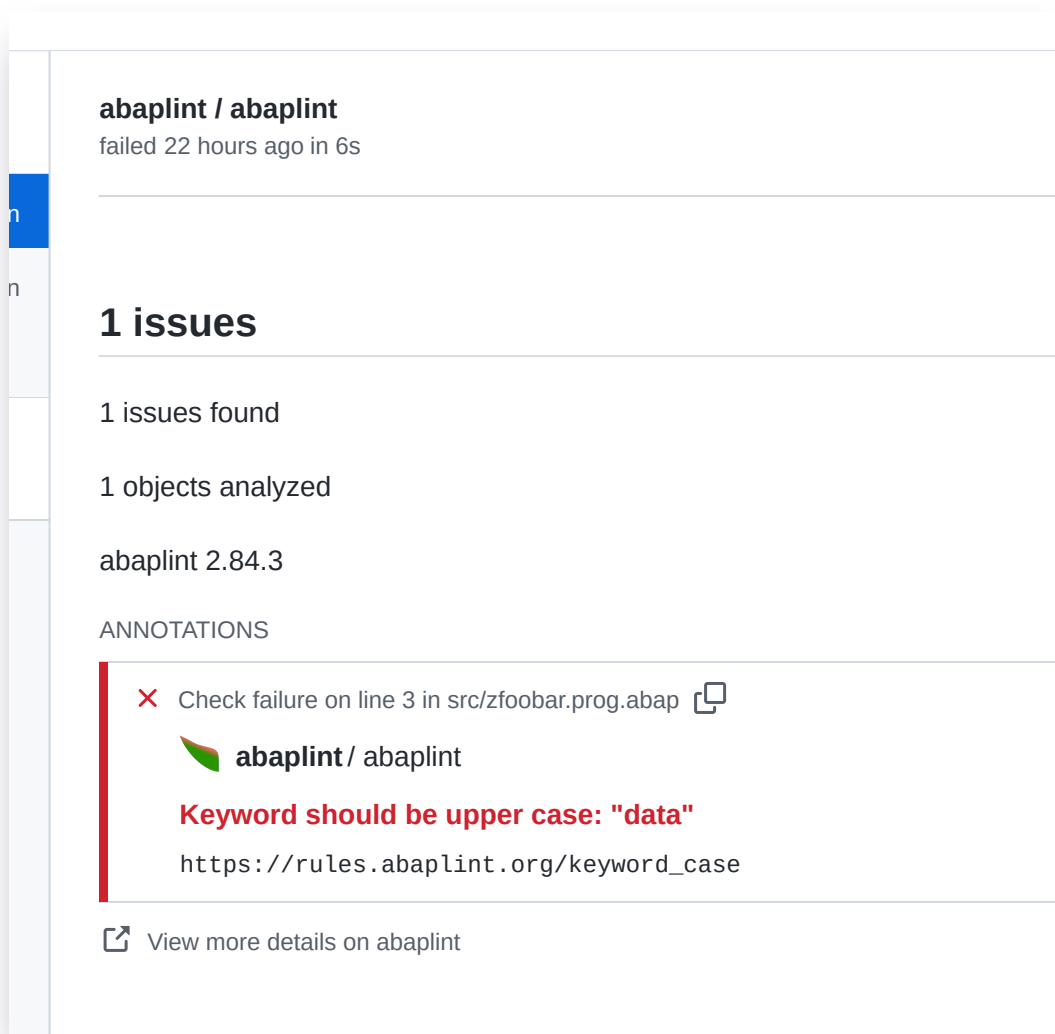
jobs:
  wait:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - name: Wait for other checks to succeed
        uses: lewagon/wait-on-check-action@v1.1.1
        with:
          ref: ${ github.event.pull_request.head.sha }
          check-name: abaplint
          repo-token: ${ secrets.GITHUB_TOKEN }
          wait-interval: 1
  unit:
    needs: wait
    runs-on: ubuntu-latest
    steps:
      - name: Echo github context
        run: echo '${ toJson(github.event) }' || true
```


4. Annotations

After installation and configuration of abaplint.app, it will report findings for each commit.



If issues are found, the "Details" link can be clicked, and the developer can check the findings directly in the pull request,

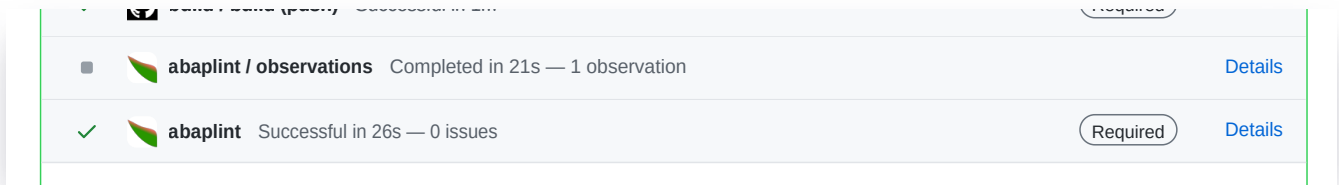


Annotations are updated after each push, and links directly to the branch which contains the change.

Additional information regarding the specific rules can be looked up on <https://rules.abaplint.org>

5. Observations

Observations are run only in the context of a pull request, and reports only "success" or "neutral" run status, so it will not block merges in case [required checks](#) are setup for branch protection.



If there are zero observations the step reports success, if some are found it reports the neutral status. A neutral status is shown in the pull request, which helps the reviewer to identify and check the observations.

For running examples see pull requests for the following repositories, <https://github.com/heliconialabs?q=observation>

5.1. New void type used

If abaplint is configured with `errorNamespace` some types are trusted by abaplint to be correct in all uses, these types are called void types.

This observation reports findings if new void types are introduced by the pull request, this helps identifying if the footprint towards eg. dependencies or the standard system increases.

abaplint / abaplint / observations
completed 21 hours ago in 21s

1 observation

No	Description
1	New void type <code>CL_GUI_CONTROL</code> used

Base: `e2a156a` , Head: `00b9846`

5.2. DB field changed

Changing structures, data elements or domains can cause transparent database tables to be changed, this observation helps making these changes visible to reviewers.

This observation is reported if:

- If the type of a field is changed
- If a field is added
- If a field is removed

If a key field is added

- If a key field is removed

2 observations

No	Description
1	Database field type changed, ZTABL1-FIELD2 , c LENGTH 1 -> c LENGTH 2
2	Database field added, ZTABL1-NEW_FIELD

Base: [11f5992](#) , Head: [85de0be](#)

5.3. Target Rules

Rules configured as `targetRules` will show up as observations, see the chapter on Target Rules.

5.4. Parallel File Changes

If identical file names are changed in more than one open pull request is reported it will trigger an observation.

Git will do its best to merge the changes, however it can be troublesome to deploy these changes to the landscape simultaneously.

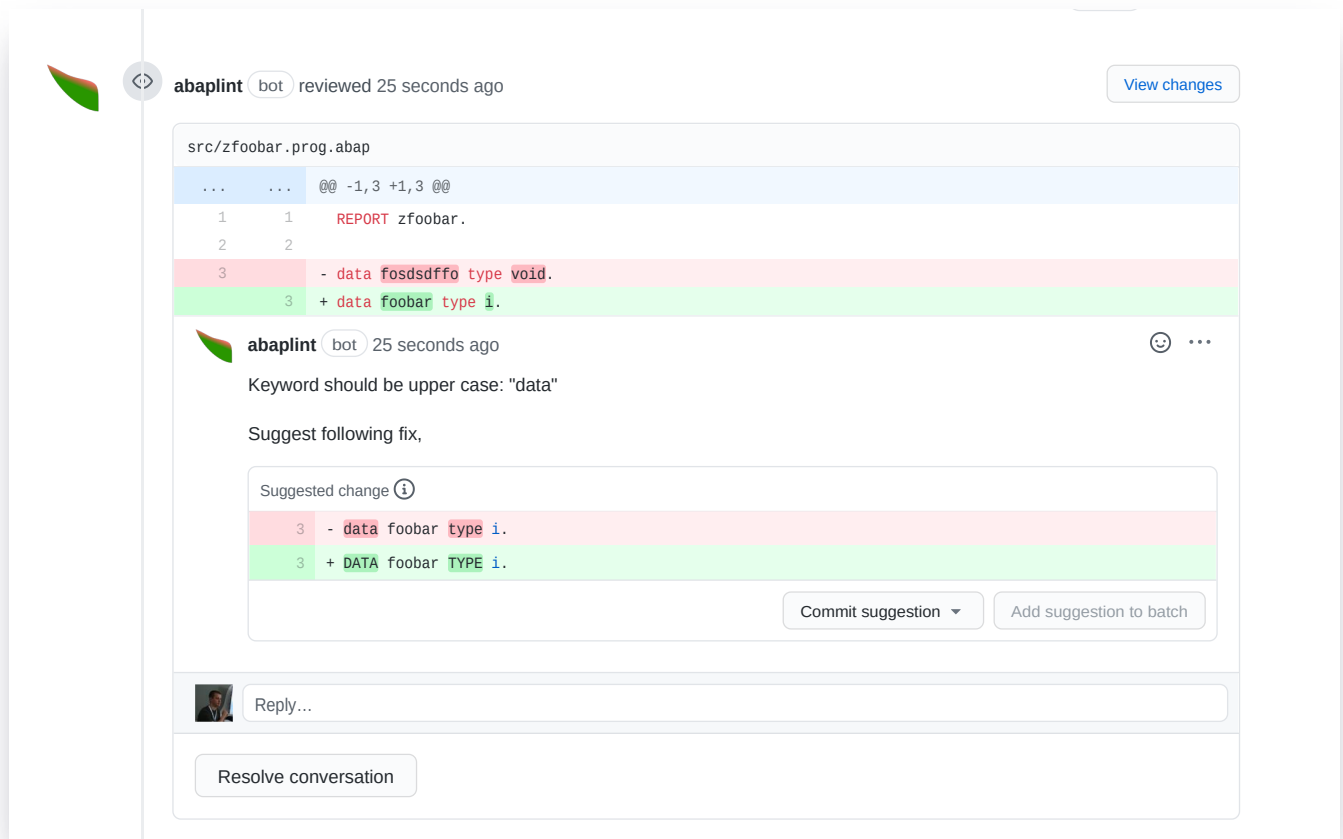
5.5. Function module type changed or deleted

If a update or remote function module is deleted, an observation is reported.

Or if its changed from update or remote to normal function module.

6. Suggestions

Suggestions by abaplint.app appear in pull requests, rules which have quick-fixes must be enabled, and only one suggestion appear at a time.



The screenshot shows a pull request interface. At the top, a comment from the **abaplint** bot, reviewed 25 seconds ago, is displayed. A [View changes](#) button is in the top right. The code snippet is from `src/zfoobar.prog.abap` and shows a diff for lines 1, 2, and 3. Line 3 has a change from `- data fosdsdffo type void.` to `+ data foobar type i.`. Below the code, the bot's message states: "Keyword should be upper case: 'data'" and "Suggest following fix,". A "Suggested change" box shows the fix: line 3, `- data` becomes `DATA`, `foobar` remains, and `type i.` becomes `TYPE i.`. At the bottom of this box are buttons for "Commit suggestion" and "Add suggestion to batch". Below the suggestion box is a "Reply..." input field. At the very bottom is a "Resolve conversation" button.

abaplint bot reviewed 25 seconds ago [View changes](#)

```
src/zfoobar.prog.abap
...    ...    @@ -1,3 +1,3 @@
1      1      REPORT zfoobar.
2      2
3      3      - data fosdsdffo type void.
          3      + data foobar type i.
```

abaplint bot 25 seconds ago 😊 ...

Keyword should be upper case: "data"

Suggest following fix,

Suggested change ⓘ

```
3      - data foobar type i.
          + DATA foobar TYPE i.
```

[Commit suggestion](#) [Add suggestion to batch](#)

 Reply...

[Resolve conversation](#)

7. Target Rules

Target rules are configured in the normal `abaplint.json` file, these rules does not set failed status, but instead allows pull requests to be merged. The target rules findings shows up as observations.

This is useful for moving the code in a better direction when its convenient. The developer and reviewer will both see the observations in the pull request status, and decide if to fix or defer. Deferring is useful for critical/complex changes, but many scenarios also allow for gradually performing fixes to move to a better target.

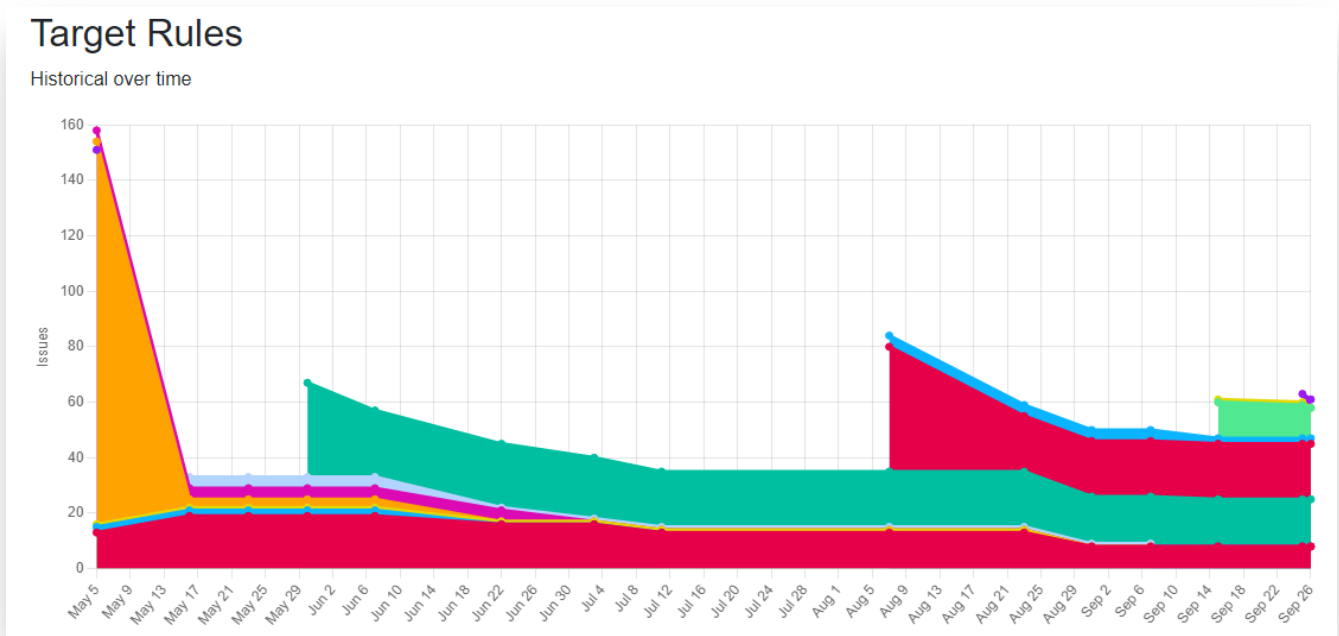
7.1. Configuration

The target rules can be defined after the normal `rules`, using the [vscode abaplint extension](#) will help with validation and intellisense.

```
{
  ...
  "rules": {
    ...
  },
  "targetRules": {
    "double_space": true,
  },
}
```

7.2. Insights

The team can always check up on the direction the code is moving. Perhaps moving some rules to required, or adding additional, or even removing if the code is not moving as first intended.



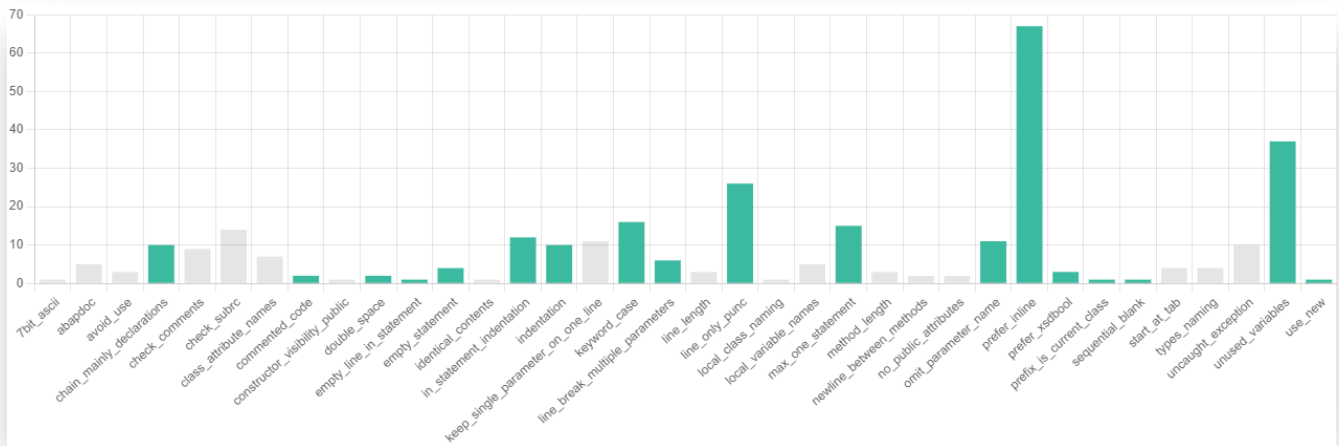
Spikes will happen when enabling additional target rules.

8. Insights

Various code insights are automatically generated daily by abaplint.app, insights can be accessed via web. Insights for public projects are public, users must login to access insights for private projects. All insights are generated only for the default branch.

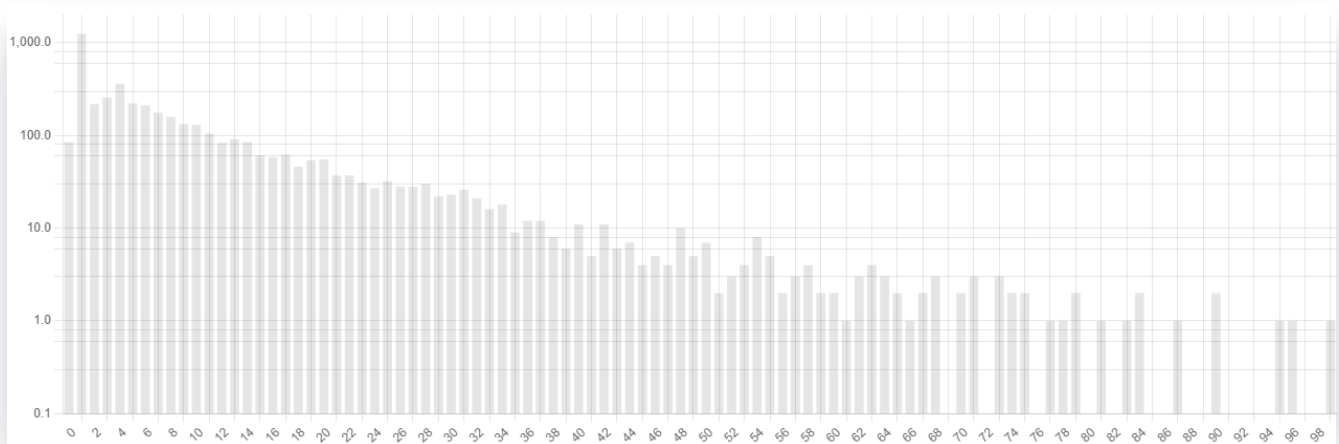
8.1. Issues

abaplint is run for the source code, if no configuration is present, the default configuration is used. Issues are listed, plus a graph gives an overview, issues with quick-fixes are shown in green,



8.2. Method Length

Method length gives an overview of the method implementation lengths across the source code. Note that the scale is logarithmic,

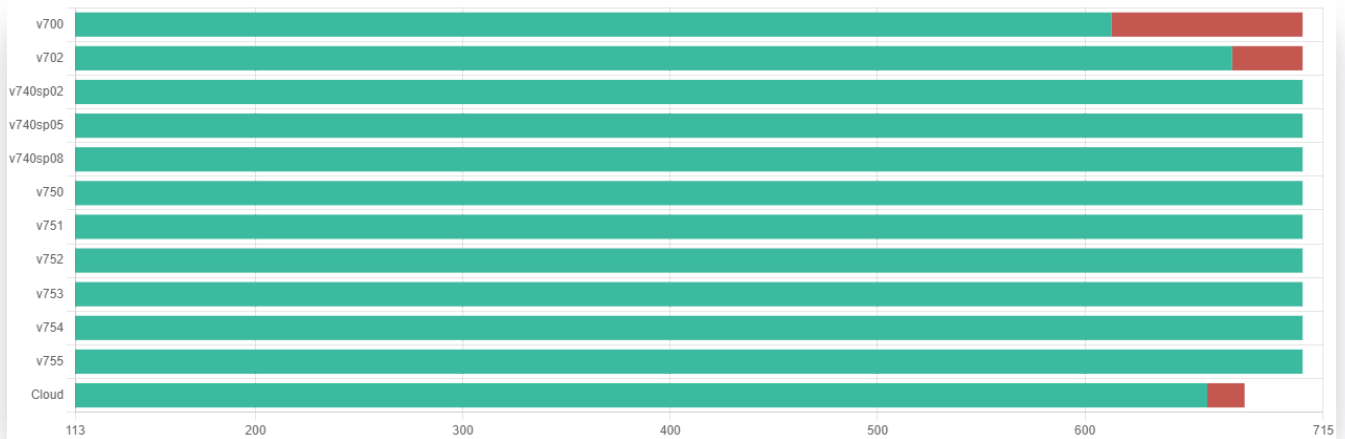


8.3. Method Complexity

Overview of method complexity measured as cyclomatic complexity.

8.4. Statement Compatibility

Statement parsing is performed against different language versions, this gives an overview of effort required to downport or upport the codebase.



8.5. Void Types

Lists void types as per the `errorNamespace` configuration, along with release, deprecation and successor information.

Name	Occurrences	Released	Deprecated	Successor
CHAR4	1			
CL_ABAP_RANDOM_INT	1			
CL_CTMENU	480			
CL_GUI_CFW	1			
FLAG	1			ABAP_BOOLEAN
NUM2	3			
NUMC2	18			
PROGNAME	1			

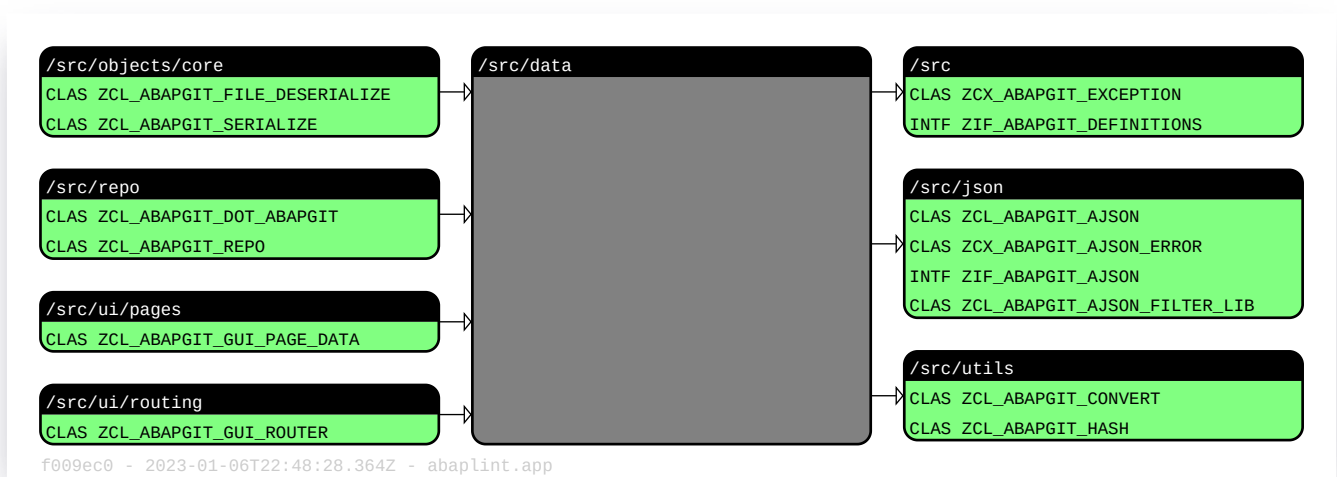
8.6. Dependencies

Lists where dependencies are used in the codebase. Dependencies are configured in the `abaplint.json` file

8.7. Package Coupling

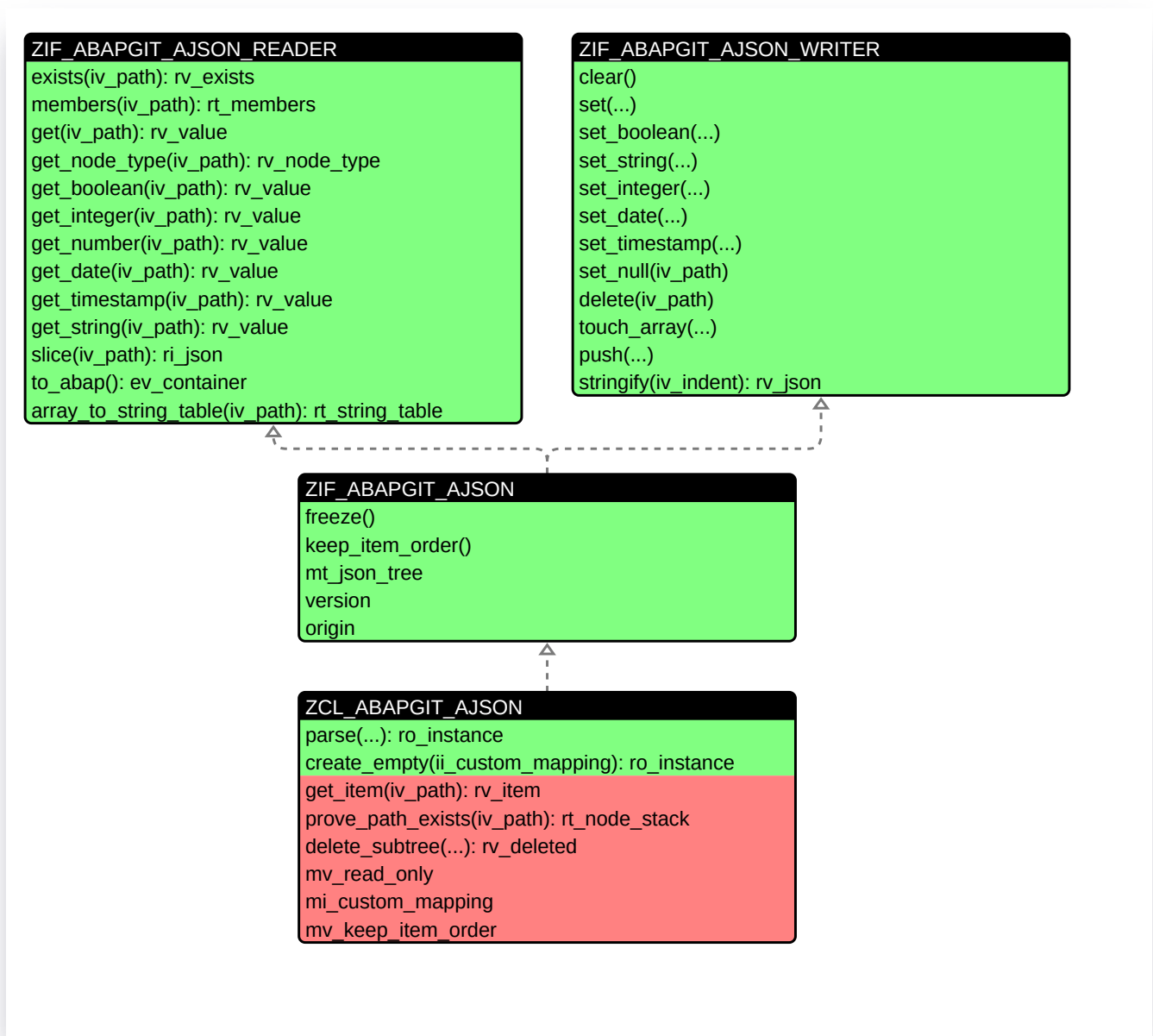
Given a folder(package) the diagram lists objects and packages using something in the package on the left side. The right side lists what is being used from the package.

Test class includes and dependencies are ignored. Dependencies into sub-folders are shown with blue background.



8.8. UML Class Diagrams

Provides auto generated UML diagrams for all global classes and interfaces, example:



8.9. Disabled Rules

If rules are disabled in the configuration, abaplint.app will try to enable to rule and provide the issue count and how much is quick-fixable.

8.10. Repository Relations

Dependencies from the abaplint configuration file is listed,

Repo Relations

Based on the dependencies defined in the default abaplint configuration file.

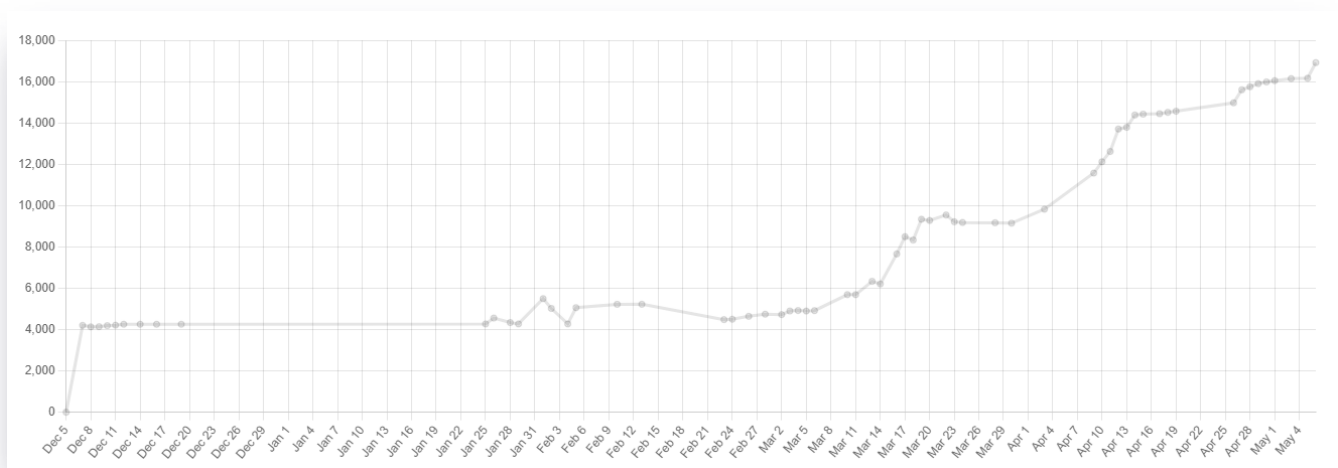
<https://github.com/abapGit/abapGit> dependency

<https://github.com/abaplint/deps> dependency

<https://github.com/larshp/ABAP-Swagger>

8.11. Lines Over Time

Show codebase size over time, including ABAP and CDS code.



8.12. Inheritance over time

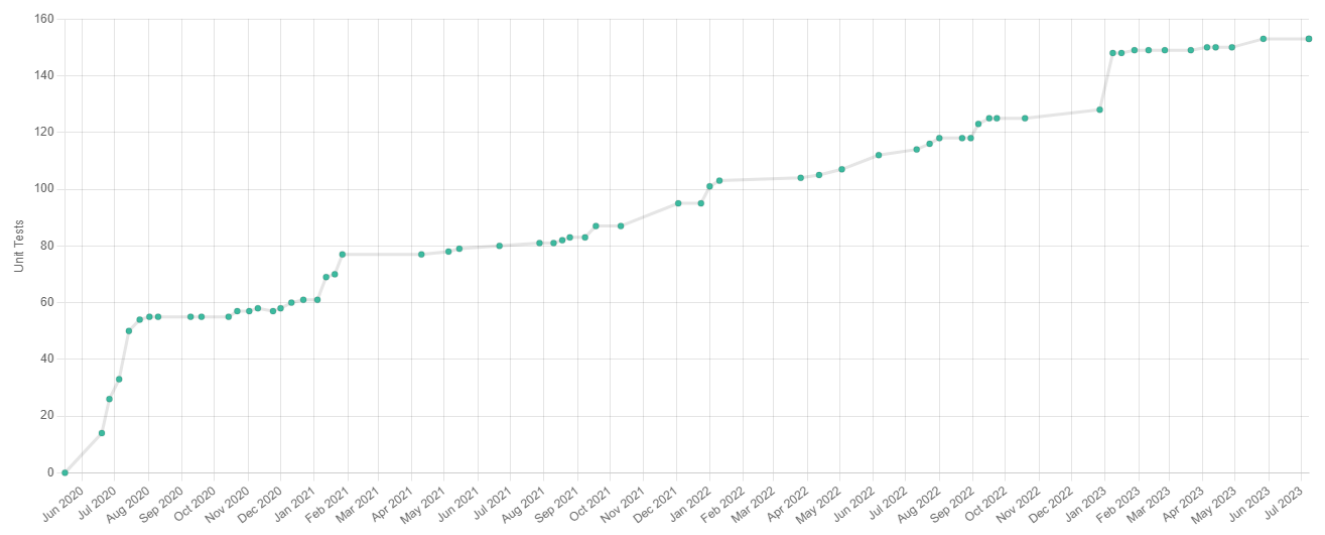
Show inheritance usage over time.

8.13. Unit Tests

Show number of unit tests over time,

Unit tests over time

Each method FOR TESTING counts as one unit test

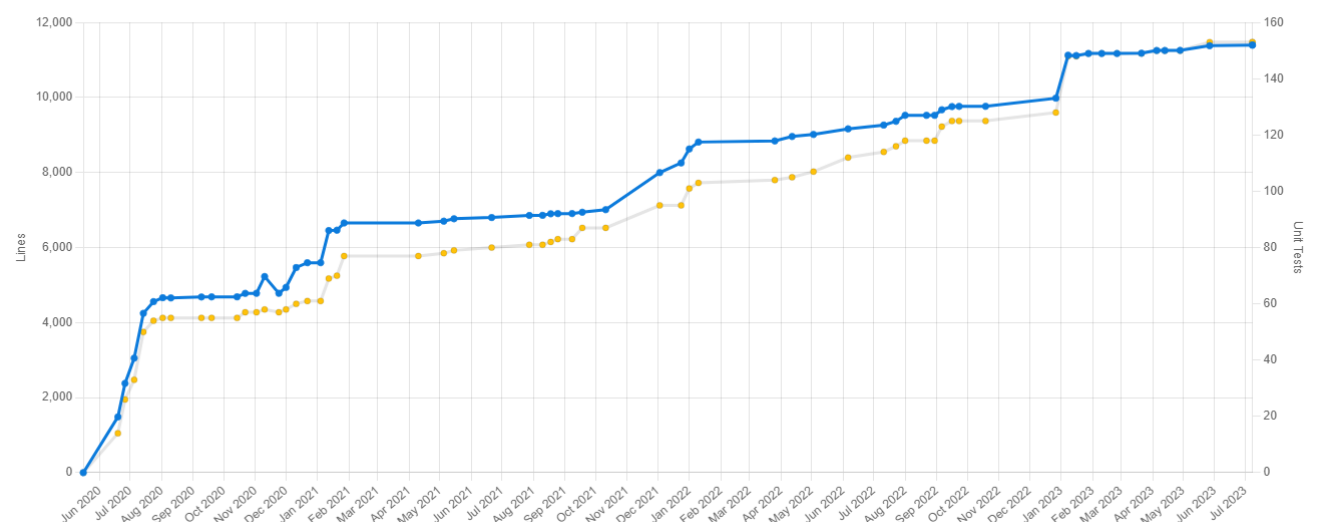


8.14. Lines vs Unit Tests

Show number of lines compared to number of unit tests over time,

Lines vs Unit Tests

Number of lines compared to number of unit tests

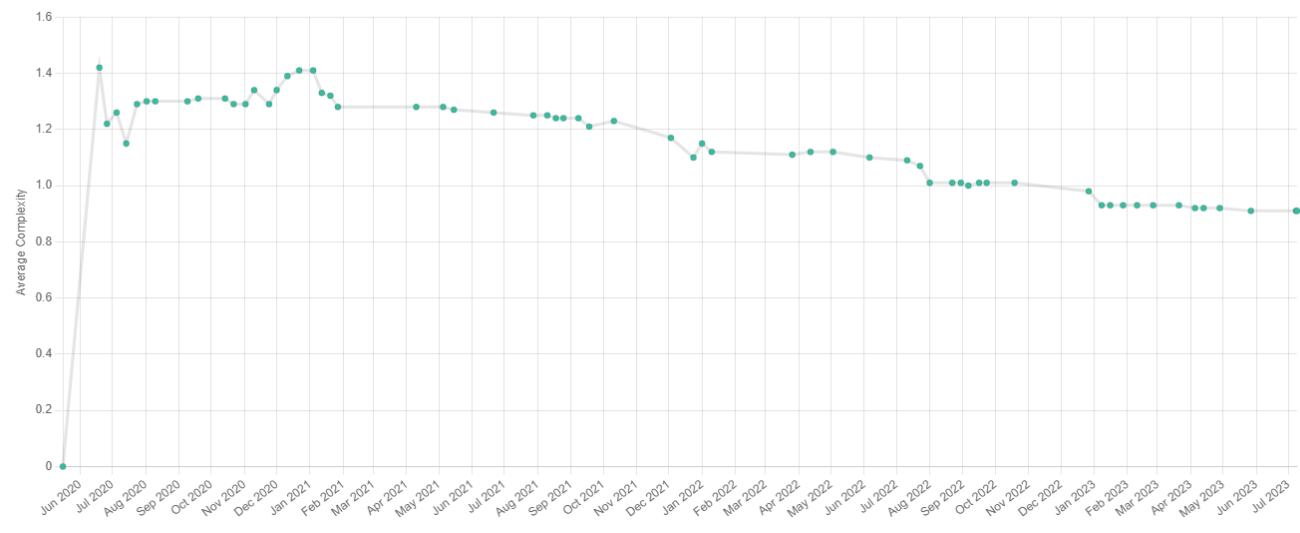


8.15. Average Complexity

Show average cyclomatic complexity over time

Average Complexity

Average cyclomatic complexity per method

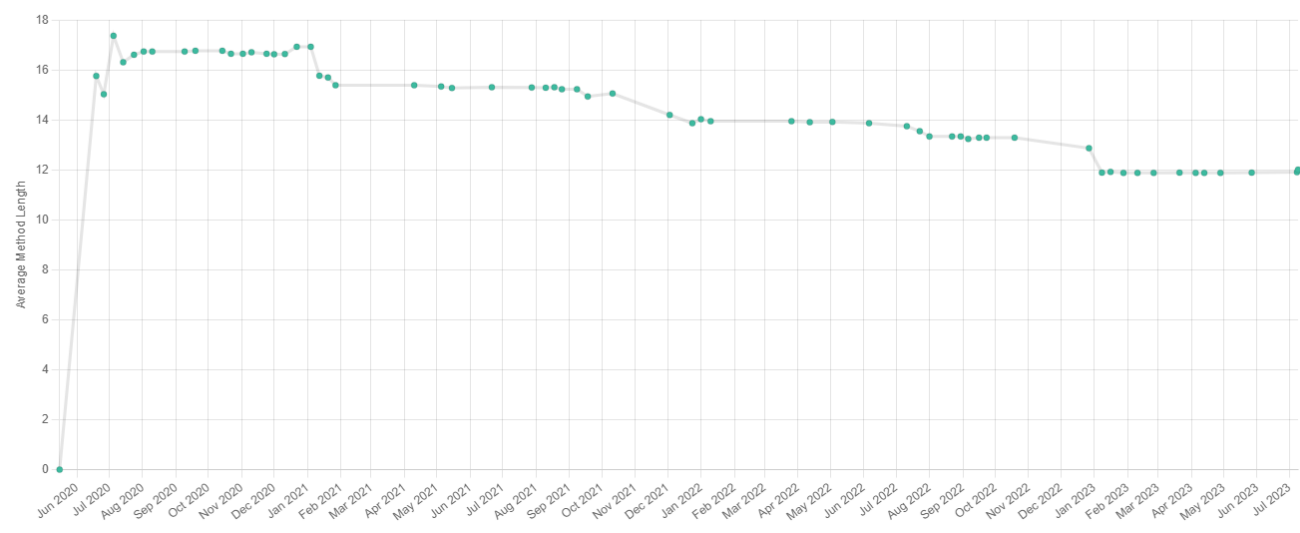


8.16. Average Method Length

Show average method length over time, measured in number of statements

Average Method Length

Average method length, number of statements



8.17. Target Rules

See the Target Rules chapter

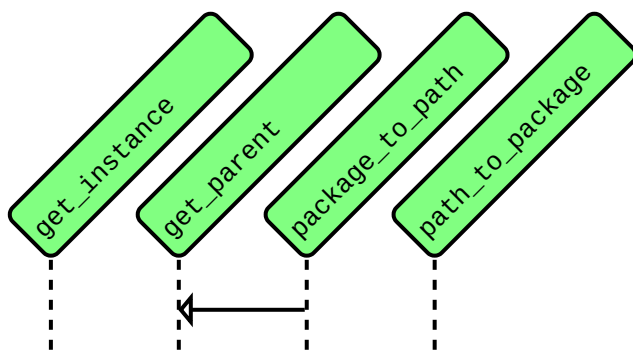
8.18. Class Length

Find the latest classes in the codebase, these are good candidates for refactoring.

8.19. Intra Class Call Graph

Display relations between methods internally in a class. The diagram does not reflect sequence, only if a method is called from another method. Only static method references are displayed.

ZCL_ABAPGIT_FOLDER_LOGIC



b82924c - 2023-10-18T22:57:07.904Z - abaplint.app

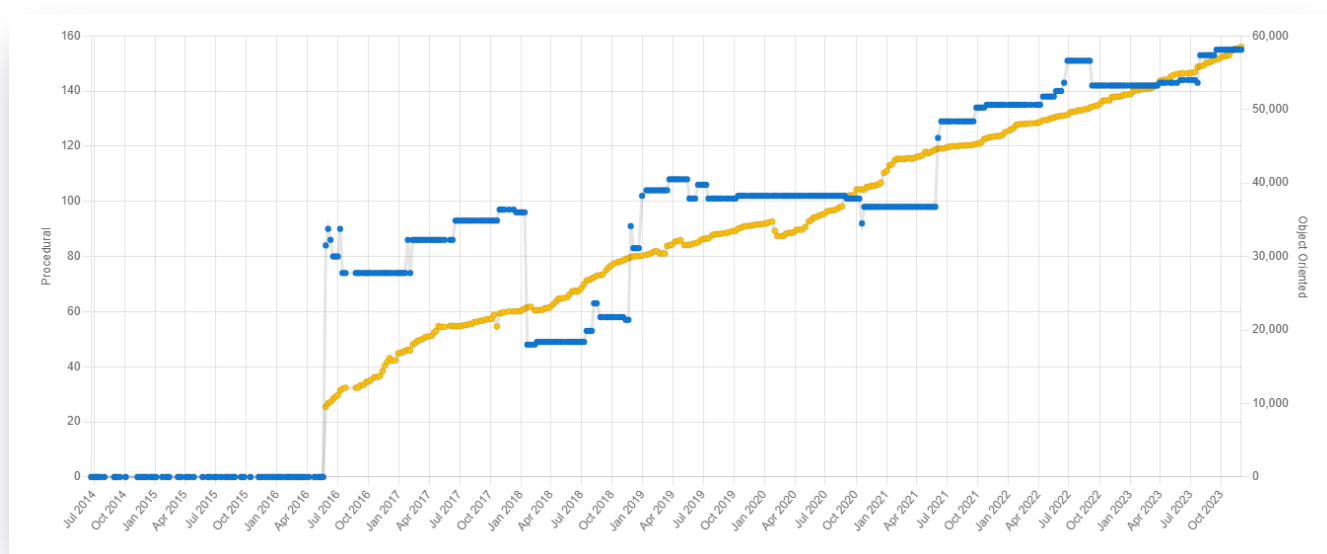
8.20. Object Types

Overview of how many objects of each type exists in the repository. Cloud indicates the object type is possible in ABAP Cloud, not that the actual object contents works in ABAP Cloud.

Type	Count	Cloud
CLAS	406	☁
INTF	87	☁
W3MI	9	
PROG	3	
FUGR	1	☁
TRAN	1	

8.21. Procedural vs OO

Number of statements in FORM + FUNCTION compared to number of statements in METHOD implementations



8.22. Table Accesses

Gives an easy overview of which database tables are accessed throughout the codebase.

8.23. API Usage

Lists usage of various SAP provided APIs throughout the codebase. Along with their public, private and steampunk status, successors and suggested successors are also listed.

8.24. Object Classifications

Overview of Clean Core classifications, use this to identify and improve code that is not Clean Core compliant.

8.25. Tree Map, Lines of Code

Visual representation of lines of code per package in the repository. The larger the box, the more lines of code in that package. This can be used to identify large packages that may need refactoring.

9. Cross Check

Some applications consists of multiple components maintained by different teams. This abaplint.app feature allows to discover syntax errors across repositories before changes are merged, so that fixes can be prepared in due time.

All repository relations and branches are found and checked one by one. If the checked already have errors, it is skipped.

Public repositories consider only public dependencies, private will take repositories in the same organization into account.

The cross check feature is enabled by default, but only reported if there are any related repositories, the feature can be disabled in `abaplint-app.json` if needed.

If there are no errors found, success is reported as a status check on the pull request. In case of new errors, a neutral status is reported, click the link from the status check to see the overview of the results,

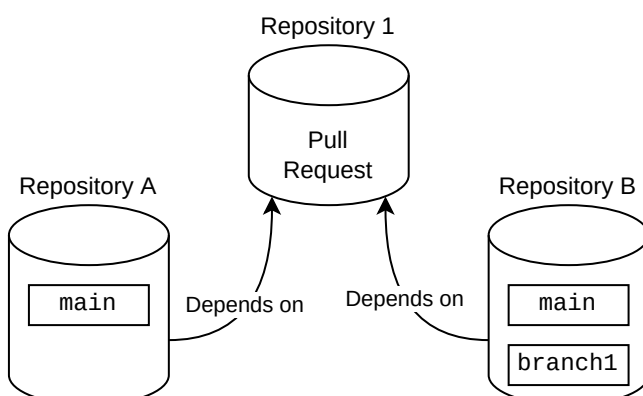
abaplint / abaplint / cross check
completed 8 minutes ago in 6m 32s

16 repos checked

Repository		
abapedia/object-existence 37 seconds	main 814cdc	
abapGit/abapGit-Plugins 19 seconds	main 42418e	

9.1. Example

A pull request is opened in "Repository 1", which have two dependent repositories: A and B,



The pull request will cross check all combinations,

- Repository A, branch `main` against the pull request
- Repository B, branch `main` against the pull request
- Repository B, branch `branch1` against the pull request

10. Additional Information

10.1. Licensing

Use in private/internal repositories requires a license purchased via [GitHub Marketplace](#).

Only active users pushing code requires licenses. Users are not named users ie. active users can change between each month with changes.

The author email address from each commit is used to count users. Use `noreply` [email addresses](#) to protect the identity of users, and align the email addresses used.

10.2. Privacy

See [Privacy Policy](#)

- To perform code analysis, the source is temporarily cloned using a temporary token
- Analysis is performed on [Digital Ocean](#) infrastructure in Germany
- Aggregated data about the source-code is stored